

JavaScript — Phase 1

The Complete Beginner's Foundation

Covering: Variables, Data Types, Operators, Conditionals, Functions, Arrays, Objects, null vs undefined, map/filter/reduce, and Loops

Prepared by Claude for Akshay

Table of Contents

1. Variables — var, let, const
2. Data Types — The 7 Primitives + Objects
3. Operators — Arithmetic, Comparison, Logical
4. Conditionals — if/else, Ternary, Switch
5. Functions — Declaration, Expression, Arrow
6. Arrays — Methods and Loops
7. Objects — Properties, Methods, Destructuring
8. var vs let vs const — Deep Comparison
9. null vs undefined — The Complete Breakdown
10. map, filter, reduce — Deep Dive
11. Practice Challenges

1. Variables

Variables are containers that store data values. Think of them as labeled boxes — you give the box a name and put something inside it.

Three Ways to Declare Variables

```
// The OLD way — avoid this
var name = "Akshay";
// The MODERN ways
let age = 25;           // value CAN change later
const city = "Nashik";  // value CANNOT change later
```

> **✔ Best Practice**
> Always start with **const**. Switch to **let** only when you need to reassign the value. Never use **var** in modern JavaScript.

Variable Naming Rules

Variable names must start with a letter, underscore (_), or dollar sign ($). They cannot start with a number. JavaScript is case-sensitive, so `myName` and `myname` are different variables. By convention, we use camelCase for variable names.

```
// Valid names
let firstName = "Akshay";
let _private = true;
let $element = "div";
let myAge2 = 25;
// Invalid names
// let 2cool = false;    // can't start with number
// let my-name = "hi";  // no hyphens allowed
```

2. Data Types

JavaScript has 7 primitive types and 1 non-primitive type. Understanding these is fundamental to everything else in JS.

The 7 Primitives

```
const str = "hello";          // String
const num = 42;               // Number
const big = 9007199254740991n; // BigInt
const bool = true;            // Boolean
const nothing = null;         // Null (intentional empty)
const notDefined = undefined; // Undefined (not yet assigned)
const id = Symbol("id");      // Symbol (unique identifier)
```

Plus the Non-Primitive

```
const obj = { key: "value" }; // Object
const arr = [1, 2, 3];        // Array (technically an Object)
const fn = function() {};     // Function (also an Object)
```

Checking Types with typeof

```
typeof "hello"     // "string"
typeof 42          // "number"
typeof true        // "boolean"
typeof undefined   // "undefined"
typeof null        // "object"   <-- Famous JS bug!
typeof {}          // "object"
typeof []          // "object"   <-- Arrays are objects!
```

> ■ **The typeof null Bug**
> `typeof null` returns `"object"` instead of `"null"`. This is a bug from the very first version of JavaScript (1995) that was never fixed because it would break existing code. To check for null, use: `value === null`

Dynamic Typing

JavaScript is dynamically typed — variables can hold any type and can change type at any time. This is both a feature and a source of bugs.

```
let value = 42;       // number
value = "hello";      // now it's a string
value = true;          // now it's a boolean
// No errors! JS doesn't care.
```

3. Operators

Arithmetic Operators

```
5 + 3      // 8   (addition)
10 - 4     // 6   (subtraction)
3 * 7      // 21  (multiplication)
15 / 4     // 3.75 (division)
10 % 3     // 1   (remainder/modulo)
2 ** 3     // 8   (exponentiation)
```

Comparison Operators — CRITICAL

```
// Loose equality (== ) — avoids type checking
5 == "5"     // true   (converts string to number)
0 == false   // true   (converts false to 0)
null == undefined  // true
// Strict equality (===) — checks type AND value
5 === "5"    // false  (different types!)
0 === false  // false  (different types!)
null === undefined  // false
```

■ Always Use === and !==

Loose equality (==) performs type coercion which leads to unexpected results. Always use strict equality (===) and strict inequality (!==) to avoid subtle bugs. This is one of the most common JavaScript interview questions.

Logical Operators

```
true && false    // false  (AND — both must be true)
true || false    // true   (OR — at least one true)
!true            // false  (NOT — flips the value)
// Short-circuit evaluation
const name = userInput || "Default";  // uses Default if userInput is falsy
const result = obj && obj.value;      // safe property access
```

Truthy and Falsy Values

In JavaScript, every value is either "truthy" or "falsy" when used in a boolean context (like an if statement). This concept is essential to understand.

```
// ALL falsy values (memorize these!):
false, 0, -0, 0n, "", null, undefined, NaN
// EVERYTHING else is truthy, including:
"0", "false", [], {}, function(){}
```

■ Truthy/Falsy Trap

Empty arrays `[]` and empty objects `{}` are TRUTHY! To check if an array is empty, use `arr.length === 0`. This is the exact bug from the practice challenge — `number % 2` returns `0` for even numbers, which is falsy.

4. Conditionals

if / else if / else

```
const score = 85;
if (score >= 90) {
  console.log("A grade");
} else if (score >= 80) {
  console.log("B grade");
} else if (score >= 70) {
  console.log("C grade");
} else {
  console.log("Keep trying");
}
```

Ternary Operator

A shorthand for simple if/else — great for assignments and inline decisions.

```
// condition ? valueIfTrue : valueIfFalse
const result = score >= 80 ? "Pass" : "Fail";
// Can nest, but don't — it gets unreadable fast
const grade = score >= 90 ? "A" : score >= 80 ? "B" : "C";
// ^ Please use if/else instead of this!
```

Switch Statement

```
const day = "Monday";
switch (day) {
  case "Monday":
  case "Tuesday":
  case "Wednesday":
  case "Thursday":
  case "Friday":
    console.log("Weekday");
    break;
  case "Saturday":
  case "Sunday":
    console.log("Weekend!");
    break;
  default:
    console.log("Invalid day");
```

```
}
```

■ Forgetting break in switch

If you forget the `break` keyword, execution "falls through" to the next case. This is sometimes useful (like grouping weekdays above) but usually a bug.

5. Functions

Functions are reusable blocks of code. JavaScript has three main ways to create them.

Function Declaration

```
function greet(name) {
  return `Hello, ${name}!`;
}
greet("Akshay");  // "Hello, Akshay!"
```

Function Expression

```
const greet = function(name) {
  return `Hello, ${name}!`;
};
greet("Akshay");  // "Hello, Akshay!"
```

Arrow Function (ES6+)

```
// Full arrow function
const greet = (name) => {
  return `Hello, ${name}!`;
};
// Short arrow (implicit return, single expression)
const greet = (name) => `Hello, ${name}!`;
// Single parameter — parentheses optional
const double = n => n * 2;
```

Default Parameters

```
const greet = (name = "World") => `Hello, ${name}!`;
greet("Akshay");  // "Hello, Akshay!"
greet();          // "Hello, World!"
```

Key Differences

Feature	Declaration	Expression	Arrow
Hoisted?	Yes	No	No
Has this?	Yes	Yes	No (inherits)
Use as method?	Yes	Yes	Avoid

✔ When to Use Which

Use **arrow functions** for most things — callbacks, array methods, short utilities. Use **function declarations** for top-level named functions that need hoisting. Avoid function expressions unless you have a specific reason.

6. Arrays

Arrays are ordered lists of values. They are one of the most-used data structures in JavaScript.

Creating and Accessing Arrays

```
const fruits = ["apple", "mango", "banana"];
fruits[0];          // "apple"   (first item)
fruits[2];          // "banana"  (third item)
fruits.length;      // 3
fruits[fruits.length - 1];  // "banana" (last item)
```

Modifying Arrays

```
const arr = [1, 2, 3];
arr.push(4);        // [1, 2, 3, 4]    add to end
arr.pop();          // [1, 2, 3]        remove from end
arr.unshift(0);     // [0, 1, 2, 3]    add to start
arr.shift();        // [1, 2, 3]        remove from start
arr.splice(1, 1);   // [1, 3]           remove at index
```

The Power Trio: map, filter, reduce

These three methods are the backbone of modern JavaScript. You will use them every single day.

```
const numbers = [1, 2, 3, 4, 5];
// map — transform each item, returns new array
const doubled = numbers.map(n => n * 2);
// [2, 4, 6, 8, 10]
// filter — keep items that pass the test
const evens = numbers.filter(n => n % 2 === 0);
// [2, 4]
// reduce — collapse array into single value
const sum = numbers.reduce((acc, n) => acc + n, 0);
// 15
```

Other Useful Array Methods

```
const arr = [1, 2, 3, 4, 5];
arr.find(n => n > 3);         // 4 (first match)
```

```
arr.findIndex(n => n > 3);    // 3 (index of first match)
arr.some(n => n > 4);         // true (at least one?)
arr.every(n => n > 0);        // true (all of them?)
arr.includes(3);              // true (contains value?)
// Chaining — combine methods
const result = [1, 2, 3, 4, 5]
  .filter(n => n % 2 === 0)
  .map(n => n * 10);
// [20, 40]
```

Looping Over Arrays

```
const fruits = ["apple", "mango", "banana"];
// for loop (classic)
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
// for...of (modern, cleaner)
for (const fruit of fruits) {
  console.log(fruit);
}
// forEach (callback style)
fruits.forEach((fruit, index) => {
  console.log(`${index}: ${fruit}`);
});
```

✔ Which Loop to Use

Use **for...of** when you just need the values. Use **forEach** for side effects. Use **.map()** when you need a new transformed array. Use a classic **for** loop when you need the index or need to break/continue.

7. Objects

Objects store data as key-value pairs. They are the foundation of almost everything in JavaScript.

Creating and Accessing Objects

```
const user = {
  name: "Akshay",
  age: 25,
  skills: ["JS", "React", "DAML"],
  greet() {
    return `Hi, I'm ${this.name}`;
  }
};
// Dot notation
user.name;          // "Akshay"
// Bracket notation (for dynamic keys)
const key = "age";
user[key];          // 25
// Method call
user.greet();       // "Hi, I'm Akshay"
```

Destructuring

Destructuring lets you unpack values from objects (and arrays) into separate variables. You will see this everywhere in React.

```
// Object destructuring
const { name, age, skills } = user;
console.log(name);    // "Akshay"
// Rename while destructuring
const { name: userName } = user;
console.log(userName); // "Akshay"
// Default values
const { country = "India" } = user;
console.log(country);  // "India" (not in object, uses default)
// Array destructuring
const [first, second] = ["a", "b", "c"];
console.log(first);   // "a"
```

Spread Operator (...)

```
// Copy an object
const copy = { ...user };
// Merge objects
const updated = { ...user, age: 26, city: "Pune" };
// Copy an array
const arrCopy = [...fruits];
// Merge arrays
const all = [...arr1, ...arr2];
```

Useful Object Methods

```
const user = { name: "Akshay", age: 25, city: "Nashik" };
Object.keys(user);     // ["name", "age", "city"]
Object.values(user);   // ["Akshay", 25, "Nashik"]
Object.entries(user);  // [["name","Akshay"], ["age",25], ...]
// Loop over object
for (const [key, value] of Object.entries(user)) {
  console.log(`${key}: ${value}`);
}
```

8. var vs let vs const — Deep Dive

Scope Comparison

```
// var is FUNCTION scoped — leaks out of blocks
function example() {
  if (true) {
    var x = 10;
  }
  console.log(x);  // 10  (leaked!)
}
// let & const are BLOCK scoped — stay contained
function example() {
  if (true) {
    let y = 10;
    const z = 20;
  }
  console.log(y);  // ReferenceError!
  console.log(z);  // ReferenceError!
}
```

Hoisting Behavior

```
// var — hoisted AND initialized as undefined
console.log(a);   // undefined (no error!)
var a = 5;
// let & const — hoisted but NOT initialized (TDZ)
console.log(b);   // ReferenceError!
let b = 5;
```

The **Temporal Dead Zone (TDZ)** is the space between the start of a block and the variable's declaration line. Accessing a let/const variable in this zone throws a ReferenceError.

Re-declaration and Re-assignment

```
// var allows re-declaration (dangerous!)
var name = "Akshay";
var name = "John";    // No error, silently overwrites
// let blocks re-declaration
let age = 25;
// let age = 30;       // SyntaxError!
age = 30;              // But re-assignment is fine
```

```
// const blocks BOTH
const city = "Nashik";
// const city = "Pune"; // SyntaxError!
// city = "Pune";       // TypeError!
```

The const Mutation Gotcha

```
const user = { name: "Akshay" };
user.name = "John";    // WORKS! Contents can change
// user = {};          // FAILS! Reference is locked
const arr = [1, 2, 3];
arr.push(4);           // WORKS! [1, 2, 3, 4]
// arr = [5, 6];       // FAILS!
```

■ const Does NOT Mean Immutable

`const` only prevents re-assignment of the variable itself. The contents of objects and arrays declared with const can still be changed. If you need true immutability, use `Object.freeze()`.

Complete Comparison Table

Feature	var	let	const
Scope	Function	Block	Block
Hoisting	Yes (as undefined)	Yes (TDZ)	Yes (TDZ)
Re-declare	Yes	No	No
Re-assign	Yes	Yes	No
Use in 2026?	Never	When needed	Default choice

9. null vs undefined — The Complete Breakdown

This is one of the most confusing parts of JavaScript. Both represent "no value" but they mean very different things.

The Simple Analogy

Think of a variable as a **box on a shelf**. **undefined** means the box exists but you never put anything in it — it's empty by default. **null** means you deliberately emptied the box — you're saying "this box intentionally has nothing."

```
let box1;              // created but never filled
console.log(box1);     // undefined
let box2 = null;       // explicitly set to empty
console.log(box2);     // null
```

When Does undefined Appear?

JavaScript gives you `undefined` automatically in these situations:

```
// 1. Variable declared but not assigned
let name;
console.log(name);           // undefined
// 2. Function with no return statement
function doNothing() {}
console.log(doNothing());    // undefined
// 3. Missing function parameter
function greet(name) {
  console.log(name);
}
greet();                      // undefined
// 4. Accessing a property that doesn't exist
const user = { name: "Akshay" };
console.log(user.age);       // undefined
```

When Does null Appear?

YOU set `null` intentionally to say "this is deliberately empty":

```
// 1. Explicitly clearing a value
let selectedUser = { name: "Akshay" };
selectedUser = null;  // "no user selected right now"
```

```
// 2. API responses often use null
const response = {
  data: null,         // "we looked, but nothing"
  error: null         // "no error occurred"
};
// 3. DOM methods return null when not found
document.getElementById("nonexistent");  // null
```

The Confusing Parts

1. typeof gives different results

```
typeof undefined   // "undefined"  (makes sense)
typeof null        // "object"     (famous JS bug!)
// How to properly check for each:
value === undefined    // check for undefined
value === null         // check for null
value == null          // catches BOTH null and undefined
```

> **■ typeof null Bug**
> `typeof null` returns `"object"` — this is a bug from JavaScript's first implementation in 1995 that was never fixed. `null` is NOT an object, it's a primitive. Always use `value === null` to check for null.

2. Equality comparisons are weird

```
null == undefined     // true   (loosely equal)
null === undefined    // false  (strictly different)
null == 0             // false
null == ""            // false
null == false         // false
// null only loosely equals undefined, nothing else!
```

3. Math behaves differently

```
Number(null)          // 0     (null becomes zero!)
Number(undefined)     // NaN   (undefined breaks math)
null + 5              // 5     (null treated as 0)
undefined + 5         // NaN   (breaks everything)
```

4. JSON handles them differently

```
JSON.stringify({ a: null, b: undefined })
// '{"a":null}'
// undefined values are REMOVED from JSON!
JSON.stringify([null, undefined])
// '[null,null]'
// In arrays, undefined becomes null
```

■ API / MongoDB Trap

When sending data to APIs or MongoDB, `undefined` fields disappear from your request body after `JSON.stringify()`. If you want to explicitly send "no value", use `null` instead.

5. Default parameters behave differently

```
function greet(name = "World") {
  return `Hello, ${name}!`;
}
greet(undefined);   // "Hello, World!"  (triggers default)
greet(null);        // "Hello, null!"   (does NOT trigger!)
```

✔ Why This Matters

`undefined` means "no value provided" so the default kicks in. `null` means "you provided a value, and that value is null" — so the default is skipped. This is a common source of bugs.

Complete Comparison Table

Feature	undefined	null
Who sets it?	JavaScript automatically	You, intentionally
Meaning	"Not yet assigned"	"Deliberately empty"
typeof	"undefined"	"object" (bug)
Number conversion	NaN	0
In JSON	Gets removed	Stays as null
Default params	Triggers default	Does NOT trigger
== null	true	true

=== null	false	true

Practical Rules

Use null when: You want to explicitly say "no value" — clearing a reference, API responses where "not found" is meaningful.

Let undefined happen naturally when: A variable isn't assigned yet, a function doesn't return anything, a property doesn't exist on an object.

```
// Never do this — pointless
let x = undefined;  // it's already undefined!
// Do this instead
let x;              // same result, cleaner
// Check for both at once
if (value == null) {
  // catches BOTH null and undefined
}
// Modern: nullish coalescing (??)
const name = user.name ?? "Default";
// uses "Default" only for null/undefined
// NOT for "" or 0 (unlike ||)
```

10. map, filter, reduce — Deep Dive

These three methods are the holy trinity of JavaScript arrays. You will use them in almost every file you write in React and Next.js. They all return new arrays and never modify the original.

map() — Transform Every Item

Takes each item, runs a function on it, returns a **new array** with the results. The original array stays untouched.

```
const numbers = [1, 2, 3, 4, 5];
const doubled = numbers.map(num => num * 2);
// [2, 4, 6, 8, 10]
console.log(numbers); // [1, 2, 3, 4, 5] — unchanged!
```

The callback receives 3 arguments:

```
const result = numbers.map((value, index, originalArray) => {
  // value — current item
  // index — current index (0, 1, 2...)
  // originalArray — the full array
  return value * 2;
});
```

Transforming object shapes (common with APIs):

```
const apiResponse = [
  { first_name: "Akshay", last_name: "S", age: 25 },
  { first_name: "John", last_name: "D", age: 30 },
];
const formatted = apiResponse.map(person => ({
  fullName: `${person.first_name} ${person.last_name}`,
  isAdult: person.age >= 18,
}));
// [{ fullName: "Akshay S", isAdult: true }, ...]
```

Real-world React example — rendering a list:

```
const users = [
  { id: 1, name: "Akshay" },
  { id: 2, name: "John" },
];
// In React JSX
```

```
{users.map(user => (
  <div key={user.id}>{user.name}</div>
))}
```

■ map() Gotcha 1 — Forgetting to return

With curly braces, you MUST use `return`. Without curly braces, the return is implicit. `numbers.map(n => { n * 2 })` returns `[undefined, undefined, ...]` because there's no return statement. Fix: `numbers.map(n => n * 2)`

■ map() Gotcha 2 — Returning objects

JS thinks `{ }` is a function body, not an object. `names.map(n => { name: n })` fails! Fix: wrap in parentheses — `names.map(n => ({ name: n }))`. This catches people ALL the time in React.

✔ map() vs forEach()

map() = transform and return a new array. **forEach()** = just do something (side effects). If you don't need the returned array, use forEach. Using map just for console.log creates a useless array of undefined values.

filter() — Keep Items That Pass a Test

Runs a test on each item. If the test returns **truthy**, the item stays. Returns a **new array**.

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const evens = numbers.filter(num => num % 2 === 0);
// [2, 4, 6, 8, 10]
const bigNumbers = numbers.filter(num => num > 5);
// [6, 7, 8, 9, 10]
```

Filtering objects (everyday use case):

```
const users = [
  { name: "Akshay", role: "developer", active: true },
  { name: "John", role: "designer", active: false },
  { name: "Sara", role: "developer", active: true },
];
// Active developers only
const activeDevelopers = users.filter(
  user => user.role === "developer" && user.active
);
// [{ name: "Akshay"... }, { name: "Sara"... }]
```

Search/filtering (common in React):

```
const searchTerm = "ak";
const results = users.filter(user =>
  user.name.toLowerCase().includes(searchTerm.toLowerCase())
);
```

Removing duplicates:

```
const numbers = [1, 2, 2, 3, 3, 3, 4];
// Using filter + indexOf
const unique = numbers.filter((num, index, arr) =>
  arr.indexOf(num) === index
);
// [1, 2, 3, 4]
// Modern way (easier):
const unique2 = [...new Set(numbers)];
// [1, 2, 3, 4]
```

■ filter() Gotcha 1 — Truthy/falsy trap

`.filter(n => n % 2)` keeps ODD numbers, not even! `4 % 2 = 0` (falsy) gets filtered OUT. Always be explicit: `n % 2 === 0`.

■ filter() Gotcha 2 — filter(Boolean) removes too much

`data.filter(Boolean)` removes ALL falsy values including `0`, `""`, and `false`. If you only want to remove null/undefined, use: `data.filter(item => item != null)`

reduce() — Collapse Array Into a Single Value

Goes through each item, carrying an "accumulator" that builds up the final result. Can return **anything** — a number, string, object, or even another array.

```
const numbers = [1, 2, 3, 4, 5];
const sum = numbers.reduce((acc, cur) => {
  return acc + cur;
}, 0);  // 0 is the initial value
// Step by step:
// acc=0,  cur=1 -> 0+1  = 1
// acc=1,  cur=2 -> 1+2  = 3
// acc=3,  cur=3 -> 3+3  = 6
// acc=6,  cur=4 -> 6+4  = 10
// acc=10, cur=5 -> 10+5 = 15
// Result: 15
```

Counting occurrences:

```
const fruits = ["apple", "banana", "apple", "mango", "apple"];
const count = fruits.reduce((acc, fruit) => {
  acc[fruit] = (acc[fruit] || 0) + 1;
  return acc;
}, {});
// { apple: 3, banana: 1, mango: 1 }
```

Grouping data (useful with API responses):

```
const users = [
  { name: "Akshay", dept: "Engineering" },
  { name: "Sara", dept: "Engineering" },
  { name: "John", dept: "Design" },
];
const grouped = users.reduce((acc, user) => {
  if (!acc[user.dept]) acc[user.dept] = [];
  acc[user.dept].push(user);
  return acc;
}, {});
// { Engineering: [...], Design: [...] }
```

Building a lookup object (super common in React):

```
const users = [
  { id: "u1", name: "Akshay" },
  { id: "u2", name: "Sara" },
```

```
];
const userMap = users.reduce((acc, user) => {
  acc[user.id] = user;
  return acc;
}, {});
// O(1) lookup instead of O(n) with .find()
userMap["u2"].name; // "Sara"
```

■ reduce() Gotcha 1 — Forgetting initial value

`[].reduce((acc, cur) => acc + cur)` throws `TypeError: Reduce of empty array with no initial value!` Always provide the second argument: `.reduce(fn, 0)` or `.reduce(fn, {})`.

■ reduce() Gotcha 2 — Forgetting to return acc

You MUST return the accumulator from the callback. Without `return acc;`, the accumulator becomes `undefined` on the next iteration and everything breaks.

■ reduce() Gotcha 3 — Wrong initial value type

If you're building an object, use `{}` as initial value, not `0`. If building an array, use `[]`. Match the initial value type to your expected output.

Chaining — The Real Power

The magic happens when you combine map, filter, and reduce together:

```
const orders = [
  { product: "Laptop", price: 1200, quantity: 1 },
  { product: "Mouse", price: 25, quantity: 3 },
  { product: "Keyboard", price: 75, quantity: 2 },
  { product: "Monitor", price: 400, quantity: 1 },
  { product: "Cable", price: 10, quantity: 5 },
];
// "Total cost of items over $50"
const total = orders
  .filter(order => order.price > 50)
  .map(order => order.price * order.quantity)
  .reduce((sum, cost) => sum + cost, 0);
// 1200 + 150 + 400 = 1750
```

```
const users = [
  { name: "Akshay", score: 85, active: true },
  { name: "John", score: 45, active: true },
  { name: "Sara", score: 92, active: false },
  { name: "Mike", score: 78, active: true },
];
// "Names of active users who passed, sorted"
const passedNames = users
  .filter(u => u.active && u.score >= 60)
  .map(u => u.name)
  .sort();
// ["Akshay", "Mike"]
```

Quick Reference

Method	Purpose	Returns	Callback returns
map()	Transform each item	New array (same length)	New value per item
filter()	Keep matching items	New array (same/shorter)	true to keep, false to drop
reduce()	Collapse into one	Anything	Updated accumulator
forEach()	Side effects only	undefined	Nothing (ignored)

✔ **When to Use Which**

Need to **transform** data? Use map(). Need to **remove** items? Use filter(). Need to **combine** everything into one result? Use reduce(). Need **side effects** only? Use forEach().

11. Practice Challenges

Test your understanding with these challenges. Try to solve them before looking at the solutions.

Challenge 1 — Basics

Challenge: Create an array of 5 numbers. Write a function that takes an array and returns a new array with only the even numbers, each multiplied by 2.

Solution

```
const numbers = [1, 2, 3, 4, 5];
const evenDoubled = (array) =>
  array
    .filter(n => n % 2 === 0)  // keep even numbers
    .map(n => n * 2);          // double each
evenDoubled(numbers);  // [4, 8]
```

■ Common Mistake

Using `.filter(n => n % 2)` without `=== 0` actually keeps ODD numbers! Since `0` is falsy in JavaScript, `4 % 2` returns `0` which gets filtered OUT. Always be explicit: `n % 2 === 0`.

Challenge 2 — map, filter, reduce Chain

Challenge: Given this array of products, write a chain that: (1) filters only in-stock items, (2) applies a 10% discount to each price, then (3) calculates the total discounted cost.

```
const products = [
  { name: "Phone", price: 699, inStock: true },
  { name: "Laptop", price: 1299, inStock: true },
  { name: "Tablet", price: 499, inStock: false },
  { name: "Watch", price: 399, inStock: true },
  { name: "Earbuds", price: 149, inStock: true },
];
```

Solution

```
const totalDiscounted = products
  .filter(p => p.inStock)              // keep in-stock only
  .map(p => p.price * 0.9)             // apply 10% discount
  .reduce((sum, price) => sum + price, 0);
// In-stock: 699, 1299, 399, 149
// After 10% discount: 629.1, 1169.1, 359.1, 134.1
// Total: 2291.4
```

✔ Breaking Down the Chain

filter removed the Tablet (inStock: false), leaving 4 items. **map** multiplied each price by 0.9 (keeping 90% = 10% discount). **reduce** summed all discounted prices into one number. Each step feeds its result into the next — that's the power of chaining!

Congratulations! You've completed Phase 1. You now understand the building blocks of JavaScript: variables, data types, operators, conditionals, functions, arrays, objects, null vs undefined, and the map/filter/reduce trio.

Next up — Phase 2: Closures, the 'this' keyword, Async JavaScript (Promises, async/await), DOM manipulation, and ES6+ features.